

Project 2:

Due Date: 12/03

Points: TBD

Rubric: TBD (Please start working on the project. Once I have set the final exam points, I will assign the total points and rubric for Project 2. It should be around 300 points, but this may change a bit. **The points mentioned in Moodle may change in the future as stated here.**)

Objective

The goal of this project is to refactor your "Job Matcher" from Project 1. You will evolve your functional prototype into a robust, maintainable, and testable software application by applying core software design principles (SOLID), documenting your architecture with UML, implementing a test suite, and documenting your work professionally.

Background

In Project 1, you successfully built a tool that scrapes or queries (API) job postings, generates embeddings for them and your resume, and calculates cosine similarity to find the best matches. This "proof of concept" focused on *functionality*.

Now, we shift our focus to *software quality*. In the real world, code is read far more often than it is written. It must be flexible to changes (e.g., swapping embedding models, adding new job sites) and verifiable (i.e., testable). This project will have you "pay down the technical debt" of Project 1 by refactoring your code to meet these professional standards.

Core Requirements

You must modify your existing Project 1 codebase to meet the following specifications.

1. Implement at least TWO SOLID Principles

You must refactor your code to clearly and correctly implement at least two of the five SOLID principles.

- **(S) Single Responsibility Principle:** A class should have only one reason to change.
- **(O) Open/Closed Principle:** Software entities should be open for extension but closed for modification.
- **(L) Liskov Substitution Principle:** Subtypes must be substitutable for their base types.
- **(I) Interface Segregation Principle:** Clients should not be forced to depend on interfaces they do not use.

- **(D) Dependency Inversion Principle:** High-level modules should not depend on low-level modules. Both should depend on abstractions.

Guidance: While you may choose any two, the **Single Responsibility Principle (SRP)** and **Dependency Inversion Principle (DIP)** are highly recommended.

- **For SRP:** Your Project 1 code likely has a single script or class that does everything (scrapes, embeds, calculates). Break this monolith into classes with single responsibilities (e.g., a JobScraper, an EmbeddingService, a SimilarityCalculator).
- **For DIP:** Your code directly calls the Gemini or OpenAI API. This is a "hard-coded dependency." Create an IEmbeddingService interface (or abstract base class) and then create a concrete GeminiEmbeddingService that *implements* it. Your main application logic should only "depend" on the interface, allowing you to "inject" the concrete service.
- **You may go for any, the above is just a suggestion.**

2. Create a UML Class Diagram

You must create a UML Class Diagram that accurately reflects the new, refactored architecture of your application. This diagram should clearly show:

- Your classes
- Their key attributes and methods
- The relationships between them (e.g., association, inheritance, dependency).
- It should be created with some software. (suggestion can use draw.io)

3. Create a UML Sequence Diagram

You must create a UML Sequence Diagram to model the dynamic behavior of one key workflow in your application. A great candidate for this is the "Find Top Job Matches" use case. Your diagram should show the sequence of method calls between your objects, starting from the initial user request to the final returned list of jobs. It should be created with some software. (suggestion can use draw.io, Mermaid online editor)

4. Write Unit Tests (I will teach this from next week)

You must write a suite of unit tests for your application.

- This is now possible because your SOLID refactoring (especially DIP) allows you to "mock" external dependencies.
- You **must** also demonstrate how to test a class that *uses* an external service (like your EmbeddingService) by mocking the interface (e.g., creating a MockEmbeddingService that returns fake data without making an API call).

5. Professional Documentation

You must provide two forms of documentation:

- **A README.md file:** This file is for a *new developer* joining your project. It must clearly explain:
 - What the project does.
 - How to set up the environment and install dependencies (e.g., `pip install -r requirements.txt`).
 - How to run the application.
 - How to run your test suite.
- **A REFACTORING.md file:** This is a brief report for me. It must explicitly state:
 - **Which two (or more) SOLID principles** you chose.
 - **Why** you chose them (i.e., what problem in your original code they solved).
 - **How** you implemented them. A "before and after" code snippet is highly effective here to demonstrate your changes.

Bonus Challenge: Dockerize Your Application

For bonus credit, containerize your application using Docker.

1. Write a Dockerfile that packages your application, its dependencies, and any necessary configuration.
2. Your README.md file should include the commands needed to build the Docker image and run the container.

Deliverables

You will submit a link to your private (in future it will be public) Git repository. The repository's main branch must contain:

1. All your refactored source code.
2. Your UML Class Diagram (e.g., as a .png or .svg file).
3. Your UML Sequence Diagram (e.g., as a .png or .svg file).
4. Your unit test files (e.g., in a /tests directory).
5. Your README.md file.
6. Your REFACTORING.md file.
7. A requirements.txt file.
8. (Bonus) Your Dockerfile.

RUBRIC: The breakdown of points will be posted soon. The most important point is as follows:

There will be an online one-on-one interview from 12/04 to 12/07. It will be a multiple-day evaluation, and you'll have to pull from GitHub to run the code. So, please do not update GitHub after the deadline, otherwise, I will deduct points. If you miss the interview, no points will be given, even if the software is working. I will ask questions related to the project. You have to upload the project to GitHub and make it private. On the day of the interview, you must make it public.

If you have any doubt please ask me during the class time, it may help other students too.